



Daniel Macharia <danielmacharia43@gmail.com>

YOHPAL LIVE RELEASE 1.0G-RC1 — Device, Observability & Final Critical Closure

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Sat, 11 Jul at 12:20

YOHPAL LIVE RELEASE 1.0G-RC1 — Device, Observability & Final Critical Closure

Release 1.0G-RC1 must remain a **strict certification release**. No new user-facing capabilities should enter this branch.

The latest verified report confirms that Release 1.0F implemented all ten planned sections, passed **32/32 tests**, fixed **7 of 13 High findings**, and removed **19 duplicate files**. However, no Android/iPhone device run occurred, Crashlytics remains unverified, environment abstraction is incomplete, rewarded ads still lack genuine server-side verification, and four High findings remain open.

report F-G .pdf

Release objective

Convert the current source-validated implementation into an immutable, device-tested release candidate eligible for a formal Release Review Board decision.

1. Freeze, Commit and Tag Release 1.0F

Required Git procedure

```
#!/usr/bin/env bash
# scripts/create_release_1_0g_rc1.sh

set -euo pipefail

EXPECTED_BRANCH="pap1"
RELEASE_BRANCH="release/yohpal-live-1.0g-rc1"
RELEASE_TAG="yohpal-live-v1.0.0-rc1"

CURRENT_BRANCH="$(git branch --show-current)"

if [[ "$CURRENT_BRANCH" != "$EXPECTED_BRANCH" ]]; then
    echo "Expected branch $EXPECTED_BRANCH but found $CURRENT_BRANCH"
    exit 1
fi

if [[ -n "$(git status --porcelain)" ]]; then
    echo "Uncommitted Release 1.0F files detected."
else
    echo "No uncommitted files found."
fi

git checkout -b "$RELEASE_BRANCH"

dart format lib test tool
flutter analyze
flutter test

git add .
git commit -m "release: complete YohPal Live 1.0F GA blocker sprint"

COMMIT_HASH="$(git rev-parse HEAD)"

git tag -a "$RELEASE_TAG" \
    -m "YohPal Live 1.0G RC1 based on $COMMIT_HASH"

echo "Branch: $RELEASE_BRANCH"
echo "Commit: $COMMIT_HASH"
echo "Tag: $RELEASE_TAG"
```

The developer must not tag while the working tree is dirty.

2. Complete Environment Configuration

The latest report confirms that Cloud Functions and LiveKit moved to `--dart-define`, while wallet, ads and subscription domains remain hardcoded.

report F-G .pdf

`lib/core/config/yohpal_environment.dart`

```
enum YohPalEnvironmentName {
  development,
  staging,
  production,
}

final class YohPalEnvironment {
  const YohPalEnvironment({
    required this.name,
    required this.functionsBaseUrl,
    required this.liveKitUrl,
    required this.walletWebUrl,
    required this.adsWebUrl,
    required this.subscriptionsWebUrl,
  });

  final YohPalEnvironmentName name;
  final Uri functionsBaseUrl;
  final Uri liveKitUrl;
  final Uri walletWebUrl;
  final Uri adsWebUrl;
  final Uri subscriptionsWebUrl;

  factory YohPalEnvironment.fromDefines() {
    final rawName = const String.fromEnvironment(
      'YOHPAL_ENV',
      defaultValue: 'development',
    );

    return YohPalEnvironment(
      name: switch (rawName) {
        'production' => YohPalEnvironmentName.production,
        'staging' => YohPalEnvironmentName.staging,
        _ => YohPalEnvironmentName.development,
      },
      functionsBaseUrl: _requiredUri(
        key: 'YOHPAL_FUNCTIONS_BASE_URL',
        value: const String.fromEnvironment(
          'YOHPAL_FUNCTIONS_BASE_URL',
        ),
      ),
      liveKitUrl: _requiredUri(
        key: 'YOHPAL_LIVEKIT_URL',
        value: const String.fromEnvironment(
          'YOHPAL_LIVEKIT_URL',
        ),
      ),
      walletWebUrl: _requiredUri(
        key: 'YOHPAL_WALLET_WEB_URL',
        value: const String.fromEnvironment(
          'YOHPAL_WALLET_WEB_URL',
        ),
      ),
      adsWebUrl: _requiredUri(
        key: 'YOHPAL_ADS_WEB_URL',
        value: const String.fromEnvironment(
          'YOHPAL_ADS_WEB_URL',
        ),
      ),
      subscriptionsWebUrl: _requiredUri(
        key: 'YOHPAL_SUBSCRIPTIONS_WEB_URL',
        value: const String.fromEnvironment(
          'YOHPAL_SUBSCRIPTIONS_WEB_URL',
        ),
      ),
    );
  }

  static Uri _requiredUri({
    required String key,
    required String value,
  }) {
    final uri = Uri.tryParse(value);

    if (value.isEmpty || uri == null || !uri.hasScheme) {
```

```

        throw StateError('Missing or invalid environment value: $key');
    }

    return uri;
}
}

```

lib/features/wallet_web/yohpal_web_handoff.dart

```

import 'package:url_launcher/url_launcher.dart';

import '../core/config/yohpal_environment.dart';

final class YohPalWebHandoff {
  const YohPalWebHandoff({
    required this.environment,
  });

  final YohPalEnvironment environment;

  Future<void> openWallet({
    required String path,
    Map<String, String>? query,
  }) async {
    await _launch(
      environment.walletWebUrl.replace(
        path: path,
        queryParameters: query,
      ),
    );
  }

  Future<void> openAds({
    required String path,
    Map<String, String>? query,
  }) async {
    await _launch(
      environment.adsWebUrl.replace(
        path: path,
        queryParameters: query,
      ),
    );
  }

  Future<void> openSubscriptions({
    required String path,
    Map<String, String>? query,
  }) async {
    await _launch(
      environment.subscriptionsWebUrl.replace(
        path: path,
        queryParameters: query,
      ),
    );
  }

  Future<void> _launch(Uri uri) async {
    final opened = await launchUrl(
      uri,
      mode: LaunchMode.externalApplication,
    );

    if (!opened) {
      throw StateError('Unable to open ${uri.host}');
    }
  }
}

```

Production command

```

flutter build appbundle \
  --release \
  --dart-define=YOHPAL_ENV=production \
  --dart-define=YOHPAL_FUNCTIONS_BASE_URL=https://us-central1-yohlab.cloudfunctions.net \
  --dart-define=YOHPAL_LIVEKIT_URL=wss://yohpal-live-ln8xib3c.livekit.cloud \
  --dart-define=YOHPAL_WALLET_WEB_URL=https://wallet.yohpal.com \
  --dart-define=YOHPAL_ADS_WEB_URL=https://ads.yohpal.com \
  --dart-define=YOHPAL_SUBSCRIPTIONS_WEB_URL=https://subscriptions.yohpal.com

```

These URLs are configuration, not secrets. Actual secret keys must remain server-side.

3. Real Rewarded-Ad Verification

Elapsed-time validation is not enough. The report explicitly records rewarded ads as untouched from a true SDK/server-verification perspective.

report F-G .pdf

pubspec.yaml

```
dependencies:  
  google_mobile_ads: ^5.3.1
```

Use a version compatible with the current Flutter/Firebase dependency graph.

lib/features/rewarded_ads/rewarded_ad_service.dart

```
import 'dart:async';  
  
import 'package:google_mobile_ads/google_mobile_ads.dart';  
  
final class RewardedAdResult {  
  const RewardedAdResult({  
    required this.completed,  
    this.rewardType,  
    this.rewardAmount,  
  });  
  
  final bool completed;  
  final String? rewardType;  
  final num? rewardAmount;  
}  
  
final class RewardedAdService {  
  RewardedAdService({  
    required this.adUnitId,  
  });  
  
  final String adUnitId;  
  
  RewardedAd? _rewardedAd;  
  
  Future<void> load() async {  
    final completer = Completer<void>();  
  
    await RewardedAd.load(  
      adUnitId: adUnitId,  
      request: const AdRequest(),  
      rewardedAdLoadCallback: RewardedAdLoadCallback(  
        onAdLoaded: (ad) {  
          _rewardedAd = ad;  
          completer.complete();  
        },  
        onAdFailedToLoad: (error) {  
          completer.completeError(  
            StateError(  
              'Rewarded ad failed to load: ${error.code}',  
            ),  
          ),  
        },  
      ),  
    );  
  
    return completer.future;  
  }  
  
  Future<RewardedAdResult> show() async {  
    final ad = _rewardedAd;  
  
    if (ad == null) {  
      throw StateError('Rewarded ad is not loaded.');    }  
  
    final completer = Completer<RewardedAdResult>();  
    var earned = false;  
    RewardItem? reward;  
  
    ad.fullScreenContentCallback = FullScreenContentCallback(  
      onAdDismissedFullScreenContent: (ad) {  
        ad.dispose();  
        _rewardedAd = null;  
  
        if (!completer.isCompleted) {  
          completer.complete(  
            RewardedAdResult(  
              completed: earned,  
              rewardType: reward?.type,  
            ),  
          );  
        }  
      },  
    );  
  
    ad.show().then(() {  
      earned = true;  
      reward = ad.reward;  
    });  
  
    return completer.future;  
  }  
}
```

```

        rewardAmount: reward?.amount,
    ),
    );
}
},
onAdFailedToShowFullScreenContent: (ad, error) {
    ad.dispose();
    _rewardedAd = null;

    if (!completer.isCompleted) {
        completer.completeError(
            StateError(
                'Rewarded ad failed to show: ${error.code}',
            ),
        );
    }
},
);

ad.show(
    onUserEarnedReward: (_, earnedReward) {
        earned = true;
        reward = earnedReward;
    },
);

return completer.future;
}
}

```

Client completion must **not** credit value directly. It should only trigger a status refresh after the ad network's server-side verification callback reaches the backend.

functions/src/rewardedAds/adMobSsvWebhook.ts

```

import { onRequest } from "firebase-functions/v2/https";
import * as admin from "firebase-admin";
import crypto from "node:crypto";

type AdMobSsvQuery = {
    ad_network?: string;
    ad_unit?: string;
    custom_data?: string;
    key_id?: string;
    reward_amount?: string;
    reward_item?: string;
    signature?: string;
    timestamp?: string;
    transaction_id?: string;
    user_id?: string;
};

function requireString(
    value: string | undefined,
    name: string,
): string {
    if (!value) {
        throw new Error(`Missing ${name}`);
    }

    return value;
}

export const adMobSsvWebhook = onRequest(
    {
        region: "us-central1",
    },
    async (request, response) => {
        try {
            const query = request.query as AdMobSsvQuery;

            const transactionId = requireString(
                query.transaction_id,
                "transaction_id",
            );
            const userId = requireString(query.user_id, "user_id");
            const rewardAmount = Number(
                requireString(query.reward_amount, "reward_amount"),
            );
            const rewardItem = requireString(
                query.reward_item,
                "reward_item",
            );
            const signature = requireString(
                query.signature,

```

```

    "signature",
  );
  const keyId = requireString(query.key_id, "key_id");

  if (!Number.isFinite(rewardAmount) || rewardAmount <= 0) {
    response.status(400).send("Invalid reward amount");
    return;
  }

  const verified = await verifyAdMobSignature({
    rawQuery: request.originalUrl.split("?")[1] ?? "",
    signature,
    keyId,
  });

  if (!verified) {
    response.status(403).send("Invalid signature");
    return;
  }

  const eventRef = admin
    .firestore()
    .collection("rewardedAdTransactions")
    .doc(transactionId);

  await admin.firestore().runTransaction(async (tx) => {
    const existing = await tx.get(eventRef);

    if (existing.exists) {
      return;
    }

    const walletRef = admin
      .firestore()
      .collection("walletBalances")
      .doc(userId);

    tx.create(eventRef, {
      transactionId,
      userId,
      rewardAmount,
      rewardItem,
      verified: true,
      keyId,
      createdAt:
        admin.firestore.FieldValue.serverTimestamp(),
    });

    tx.set(
      walletRef,
      {
        rewardBalance:
          admin.firestore.FieldValue.increment(
            rewardAmount,
          ),
        updatedAt:
          admin.firestore.FieldValue.serverTimestamp(),
      },
      { merge: true },
    );
  });

  response.status(200).send("OK");
} catch (error) {
  console.error("AdMob SSV failure", error);
  response.status(400).send("Invalid request");
}
},
);

async function verifyAdMobSignature(input: {
  rawQuery: string;
  signature: string;
  keyId: string;
}): Promise<boolean> {
  // Production implementation must:
  // 1. Fetch and cache Google's published AdMob SSV public keys.
  // 2. Select the key matching keyId.
  // 3. Remove signature and key_id from the signed query material
  //    exactly as required by Google's SSV specification.
  // 4. Verify the signature using the prescribed algorithm.
  //
  // Never return true without actual cryptographic verification.
  void crypto;
  void input;

  throw new Error(

```

```

    "AdMob public-key verification must be implemented " +
    "against the official SSV specification before deploy.",
  );
}

```

The webhook above intentionally refuses to approve rewards until real cryptographic verification is implemented. A placeholder returning true would recreate the same fraud exposure.

4. Crashlytics Device Proof

Hidden verification action

```

import 'package:firebase_crashlytics/firebase_crashlytics.dart';

final class CrashlyticsVerificationService {
  const CrashlyticsVerificationService();

  Future<void> sendNonFatalVerification() async {
    await FirebaseCrashlytics.instance.setCustomKey(
      'release_candidate',
      '1.0G-RC1',
    );

    await FirebaseCrashlytics.instance.recordError(
      StateError('YohPal 1.0G-RC1 verification event'),
      StackTrace.current,
      fatal: false,
      reason: 'Release candidate observability proof',
    );
  }

  Future<void> triggerFatalVerification() async {
    await FirebaseCrashlytics.instance.setCustomKey(
      'release_candidate',
      '1.0G-RC1',
    );

    FirebaseCrashlytics.instance.crash();
  }
}

```

Required evidence:

- Android non-fatal event screenshot.
 - Android fatal event screenshot.
 - iPhone non-fatal event screenshot.
 - iPhone fatal event screenshot.
 - Commit hash displayed as a Crashlytics custom key.
-

5. Password Reset and SSO Device Test Harness

integration_test/auth_device_test.dart

```

import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets(
    'password reset screen accepts valid email',
    (tester) async {
      // Launch app using the project's normal test bootstrap.
      // Navigate to Login → Forgot password.
      // Enter a designated QA email.
      // Submit and assert the confirmation screen appears.
      //
      // Email delivery and reset-link completion must still be
      // confirmed manually because they cross application boundaries.
    },
  );
}

```

SSO evidence cannot be established by source review alone. The report explicitly states that neither SSO path nor the other device-dependent features were exercised on a device.

report F-G .pdf

Required:

- Google login on Android.
 - Google login on iPhone.
 - Apple login on a physical iPhone.
 - Returning-user login.
 - Cancelled OAuth flow.
 - Disabled provider failure.
 - Logout and second-account login.
-

6. Close the Four Remaining High Findings

A. AI publish checklist enforcement

```
enum AiPublishStatus {
  notRequested,
  pending,
  approved,
  rejected,
}

final class PublishEligibility {
  const PublishEligibility({
    required this.aiEnhancementRequested,
    required this.aiPublishStatus,
  });

  final bool aiEnhancementRequested;
  final AiPublishStatus aiPublishStatus;

  bool get canPublish {
    if (!aiEnhancementRequested) return true;
    return aiPublishStatus == AiPublishStatus.approved;
  }

  String? get blockingReason {
    if (canPublish) return null;

    return switch (aiPublishStatus) {
      AiPublishStatus.pending =>
        'AI enhancement is still being processed.',
      AiPublishStatus.rejected =>
        'AI enhancement must be corrected before publishing.',
      _ => 'Complete the AI enhancement workflow.',
    };
  }
}

final eligibility = PublishEligibility(
  aiEnhancementRequested: controller.aiEnhancementSelected,
  aiPublishStatus: controller.aiPublishStatus,
);

FilledButton(
  onPressed: eligibility.canPublish
    ? controller.publish
    : null,
  child: const Text('Publish'),
)
```

B. Creator avatar upload

```
import 'dart:io';

import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_storage/firebase_storage.dart';
import 'package:image_picker/image_picker.dart';

final class CreatorAvatarService {
  CreatorAvatarService({
    FirebaseAuth? auth,
    FirebaseStorage? storage,
    ImagePicker? picker,
  }) : _auth = auth ?? FirebaseAuth.instance,
       _storage = storage ?? FirebaseStorage.instance,
       _picker = picker ?? ImagePicker();

  final FirebaseAuth _auth;
  final FirebaseStorage _storage;
  final ImagePicker _picker;
```



```

Future<Uri?> pickAndUpload() async {
  final uid = _auth.currentUser?.uid;

  if (uid == null) {
    throw StateError('User must be signed in.');
```

C. Chat send failure UI

```

Future<void> sendMessage() async {
  final text = messageController.text.trim();

  if (text.isEmpty || sending) return;

  setState(() {
    sending = true;
    sendError = null;
  });

  try {
    await chatController.sendMessage(text);
    messageController.clear();
  } catch (error, stack) {
    await crashReporter.recordError(
      error,
      stack,
      reason: 'Chat message send failure',
    );

    if (!mounted) return;

    setState(() {
      sendError = 'Message failed to send.';
      failedMessage = text;
    });
  } finally {
    if (mounted) {
      setState(() => sending = false);
    }
  }
}

if (sendError != null)
  MaterialBanner(
    content: Text(sendError!),
    actions: [
      TextButton(
        onPressed: sendMessage,
        child: const Text('Retry'),
      ),
    ],
  )
)
```

D. Cross-account state cleanup

```

abstract interface class SessionScopedState {
  Future<void> clearSession();
}

final class YohPalSessionCleanupService {
  const YohPalSessionCleanupService({
    required this.stores,
  });
}
```

```
final List<SessionScopedState> stores;

Future<void> clear() async {
  final failures = <Object>[];

  for (final store in stores) {
    try {
      await store.clearSession();
    } catch (error) {
      failures.add(error);
    }
  }

  if (failures.isNotEmpty) {
    throw StateError(
      'Failed to clear ${failures.length} session stores.',
    );
  }
}

Future<void> logout() async {
  loading = true;
  notifyListeners();

  try {
    await cleanupService.clear();
    await repository.signOut();
  } finally {
    loading = false;
    notifyListeners();
  }
}
```

- Stores should include:
- Feed state.
 - Wallet state.
 - Notification-token binding.
 - Creator-profile state.
 - Search history where account-specific.
 - Chat subscriptions.
 - Player/controller pools containing user-specific content.

7. Android Validation Matrix

Required device evidence:

Test	Required result
Release installation	Pass
Password reset email	Delivered and completed
Google Sign-In	Pass
Logout and second account	No leaked state
Search navigation	Pass
Live Discovery	Pass
Feed retry	Pass

Feed pagination	Pass
Mute	Pass
Double-tap like	Pass
Live reconnect	Pass after network interruption
Avatar upload	Pass
Chat failure retry	Pass
Wallet return	Pass
Crashlytics	Event visible
Rewarded ad	Server-verified reward only

8. iPhone Validation Matrix

Required:

- iOS 15 or newer physical device.
- Password-reset round trip.
- Google Sign-In.
- Apple Sign-In.
- Crashlytics event.
- Live reconnect under Wi-Fi interruption.
- VoiceOver check.
- Wallet deep-link return.
- Avatar upload.
- Logout state cleanup.
- 30-minute feed playback.
- 30-minute live stream.

9. Device Evidence Model

`tool/release_certification/device_result.dart`

```
enum DeviceTestStatus {
  passed,
  failed,
  blocked,
}

final class DeviceTestResult {
  const DeviceTestResult({
    required this.id,
    required this.platform,
    required this.device,
    required this.osVersion,
    required this.buildNumber,
    required this.status,
    required this.evidencePath,
    required this.executedAt,
    required this.testers,
    this.notes,
  });
```

```

final String id;
final String platform;
final String device;
final String osVersion;
final String buildNumber;
final DeviceTestStatus status;
final String evidencePath;
final DateTime executedAt;
final String tester;
final String? notes;

Map<String, Object?> toJson() => {
  'id': id,
  'platform': platform,
  'device': device,
  'osVersion': osVersion,
  'buildNumber': buildNumber,
  'status': status.name,
  'evidencePath': evidencePath,
  'executedAt': executedAt.toUtc().toIso8601String(),
  'tester': tester,
  'notes': notes,
};
}

```

10. RC1 Certification Commands

```

flutter clean
flutter pub get
dart format --output=none --set-exit-if-changed lib test tool
flutter analyze
dart run tool/check_routes.dart
flutter test --coverage

cd functions
npm ci
npm run build
npm test

```

Required builds:

```

flutter build appbundle --release \
  --dart-define-from-file=config/production.json

flutter build ipa --release \
  --dart-define-from-file=config/production.json

```

config/production.json

```

{
  "YOHPAL_ENV": "production",
  "YOHPAL_FUNCTIONS_BASE_URL": "https://us-central1-yohlab.cloudfunctions.net",
  "YOHPAL_LIVEKIT_URL": "wss://yohpal-live-ln8xib3c.livekit.cloud",
  "YOHPAL_WALLET_WEB_URL": "https://wallet.yohpal.com",
  "YOHPAL_ADS_WEB_URL": "https://ads.yohpal.com",
  "YOHPAL_SUBSCRIPTIONS_WEB_URL": "https://subscriptions.yohpal.com"
}

```

Do not commit files containing secrets. This file should contain only public client configuration.

11. Release Review Board Decision Rules

GO

Require:

- No open or partially closed Critical finding.
- Real rewarded-ad SSV operational.
- Crashlytics proven on Android and iPhone.
- Environment abstraction complete.
- All four remaining High findings closed.
- Android and iPhone matrices pass.
- CI green against the tagged commit.
- Signed evidence package complete.

CONDITIONAL GO

Permitted only when:

- All Critical findings are fully passed.
- Remaining non-financial High findings are bounded by feature flags.
- Rollout is restricted to 1–5%.
- Mission Control alerts and rollback are active.

NO-GO

Mandatory when:

- Any Critical finding remains partial, unverified or open.
- Reward crediting can occur without SSV.
- Crashlytics is unproven.
- Either physical device validation fails.
- The release candidate lacks a commit/tag.
- Evidence does not match the tagged commit.

Required Developer Output

1. Release branch
2. Commit hash
3. RC1 tag
4. Clean working-tree proof
5. Complete environment-config proof
6. Ad SDK integration proof
7. Server-side ad verification proof
8. Android device report
9. iPhone device report
10. Crashlytics Android proof
11. Crashlytics iPhone proof
12. Password-reset proof
13. Google SSO proof
14. Apple SSO proof
15. AI publish gate proof
16. Avatar upload proof
17. Chat failure/retry proof
18. Cross-account cleanup proof
19. flutter analyze output
20. flutter test output and coverage
21. Cloud Functions build/test output
22. CI workflow proof
23. Updated Production Readiness Review
24. Signed Release Review Board resolution

Current Executive Position

Source-level progress: PASS.

Release-candidate readiness: HOLD.

General Availability: NO-GO until RC1 evidence passes.

This follows the report's central conclusion: the implementation has advanced considerably, but no commit, tag or physical-device validation has yet been produced, and several Critical/High closures remain incomplete.

report F-G .pdf

Next Smart Move

Execute **YOH PAL LIVE RELEASE 1.0G-RC1 VALIDATION REPORT** after developers submit the tagged commit and full Android/iPhone evidence package; issue **GO**, **CONDITIONAL GO**, or **NO-GO** without starting another implementation release.